

Python programming

Why this unit is taught the way it is

What this unit teaches, and where it ends

13 lessons: 13 with an online experience, 0 paper assessments and 0 bonus challenges. 627 activities in all, worth 1242 marks.

By the end of it a pupil should be able to:

- know how to print lines of output in the order a task asks for [pr-101](#)
- know how to hold a value in a variable or a constant, and how to read one with input() [pr-101](#)
- understand why a value that has been typed in must be cast before it can be calculated with [pr-101](#)
- know which arithmetic operator each kind of calculation needs [pr-102](#)
- understand the difference between /, // and % and what each one hands back [pr-102](#)
- know how to round a result, and how to draw a random number from a seed [pr-102](#)
- know how to write a condition using the comparison operators [pr-103](#)
- understand how if, elif and else choose between several outcomes [pr-103](#)
- know how to join two conditions together with and, or and not [pr-103](#)
- know how to write a for loop that repeats a block a set number of times [pr-104](#)
- understand how range() decides which values the loop counter takes [pr-104](#)
- know how to build a running total inside a loop, and how to put one loop inside another [pr-104](#)
- know how to write a while loop and what makes it stop repeating [pr-105](#)
- understand how a validation loop keeps asking until the value typed in is acceptable [pr-105](#)
- know how a sentinel value ends a loop when the amount of data is not known in advance [pr-105](#)
- know how to find the length of a string and the character at a given position [pr-106](#)
- understand how a slice takes part of a string, and why its end position is left out [pr-106](#)
- know how to change case, search inside text, and move between a character and its code [pr-106](#)
- know how to create a list and read one item out of it by its index [pr-107](#)
- understand how to traverse a list in order to total, count or compare its items [pr-107](#)
- know how to add an item onto a list and how to take one out again [pr-107](#)
- know how to build a two-dimensional list as a list of lists and fill it from input [pr-108](#)
- understand how a row index and a column index together pick out one cell [pr-108](#)
- know how to write nested loops that print a grid and total any row or column of it [pr-108](#)
- know how to define a subprogram with def and how to call it [pr-109](#)
- understand how a parameter carries a value into a subprogram and how return carries one back out [pr-109](#)
- know how to write a subprogram that uses a variable of its own, and one that changes a shared variable [pr-109](#)
- know how to open a text file, read what is in it and close it again [pr-110](#)
- understand what write mode does to a file, and how append mode differs from it [pr-110](#)
- know how to split one line of a file into its separate fields and use them [pr-110](#)
- know how to code a linear search and a binary search, and report the position each one finds [pr-111](#)
- understand how a bubble sort, an insertion sort and a merge sort each put a list in order [pr-111](#)
- know how to count the comparisons an algorithm makes, in order to compare two of them [pr-111](#)
- know how to code a presence check, a type check and a range check [pr-112](#)
- understand how a length check and a format check differ, and why the length is tested first [pr-112](#)

- know how to write a validation loop, and an authentication check that allows a set number of attempts [pr-112](#)
- know how SELECT and FROM choose the fields and the table a query reads [pr-113](#)
- understand how WHERE decides which records a query hands back [pr-113](#)
- know how to join conditions with AND and OR, and how to put results in order with ORDER BY [pr-113](#)

Why the lessons are in this order

Each lesson's starter retrieves the one before it, so the order is the retrieval schedule as much as the teaching one. The chain for this unit:

LESSON	TEACHES	RETRIEVES
1 Printing and variables	know how to print lines of output in the order a task asks for	nothing before it in this unit
2 Arithmetic	know which arithmetic operator each kind of calculation needs	Printing and variables
3 Selection	know how to write a condition using the comparison operators	Arithmetic
4 Count-controlled loops	know how to write a for loop that repeats a block a set number of times	Selection
5 Condition-controlled loops	know how to write a while loop and what makes it stop repeating	Count-controlled loops
6 String handling	know how to find the length of a string and the character at a given position	Condition-controlled loops
7 Lists	know how to create a list and read one item out of it by its index	String handling
8 Two-dimensional lists	know how to build a two-dimensional list as a list of lists and fill it from input	Lists
9 Subprograms	know how to define a subprogram with def and how to call it	Two-dimensional lists
10 File handling	know how to open a text file, read what is in it and close it again	Subprograms
11 Searching and sorting	know how to code a linear search and a binary search, and report the position each one finds	File handling
12 Validation and authentication	know how to code a presence check, a type check and a range check	Searching and sorting
13 SQL queries	know how SELECT and FROM choose the fields and the table a query reads	Validation and authentication

What pupils find hard here

These are the misconceptions the unit's own teaching names and works against. They are not a general list: each one is written on a wall of the experience it belongs to.

- leaving the quotation marks off, so Python hunts for a variable with that name instead. [pr-101](#)
- a second assignment replaces the value, it does not add to it. [pr-101](#)
- reading two answers into the same variable, so the first one is lost. [pr-101](#)
- joining a number to a string with +, which Python refuses to do. [pr-101](#)
- adding to what input() handed back without casting it first, which joins the digits on the end instead. [pr-101](#)
- printing the lines in a different order from the one the task asked for. [pr-101](#)
- writing 2n for two lots of n, when Python needs 2 * n. [pr-102](#)
- using / when the answer must be whole, which leaves a .0 on the end. [pr-102](#)
- writing 5 ^ 2, which does something else entirely in Python. [pr-102](#)
- rounding early, then carrying the rounded value into the next calculation. [pr-102](#)
- calling randint again instead of storing the number you have already drawn. [pr-102](#)

- leaving brackets out, so Python works the sum out in a different order from the one you meant. pr-102
- comparing with one =, when one = stores a value and two compare them. pr-103
- missing the colon, or not indenting the block underneath it. pr-103
- putting a wide band first, so it catches everything and the later branches never run. pr-103
- writing `n == 3 or 7`, which Python reads as something quite different. pr-103
- nesting where a single and would have said the same thing far more clearly. pr-103
- using `>` where the rule says 'or more', which loses the boundary value. pr-103

Every multiple-choice and Predict activity in the unit — 109 of them — offers wrong answers that are the misconception, not filler. The split across those options at the hinge question is the cheapest diagnostic in the lesson.

Where the explaining, modelling and practising happen

PHASE WHERE IT IS

Retrieve	The starter on the worksheet, before any device is opened. It is answered on paper and corrected before the lesson moves on.
Explain	The slides and the station walls carry the same explanation, so a teacher can teach from the front and a pupil can re-read it in the room.
Model	The worked example on the early activities of each station - 314 of them in this unit. Each uses different data from the task on purpose, so copying it does not finish the task.
Practise	191 guided activities, then 436 independent ones.
Check	The badge questions mark as the pupil goes; the hinge question is for the class at once.
Apply	78 challenge activities, and the exam practice on the worksheet.
Review	The exit question and the confidence columns. The confidence columns supplement the evidence; they do not stand in for it.

How the website, the slides and the worksheet divide the work

They are not three copies of the same thing.

- **The slides** are for the front of the room: the explanation and the model, before anyone is working alone.
- **The experience** is where a pupil reads, decides and is marked. It gives the immediate feedback a teacher cannot give thirty pupils at once.
- **The worksheet** is the record and the writing. The key fact in their own words, the station challenge, the exam practice. It is also what survives when the devices do not.

The rule that holds them together is **write first, then answer at the badge**. A pupil who taps first has turned the lesson into a quiz game, and the worksheet is how you can see at a glance who did.

Why a 360° room, and what it should produce

The room is not there to be impressive. It is there because this unit's content is a set of related parts that a pupil has to hold together, and putting one part on each wall makes the set visible and navigable: a pupil can turn back to station 2 while answering station 4. The evidence it should produce is on paper — 78 key facts and 78 station challenges in a pupil's own words — and in the score screen, which shows which station a pupil lost marks on rather than one number.

No claim is made here that immersion improves learning on its own. What the format does is make the structure of a topic physical and keep a pupil's attention on one part at a time; the teaching is in the explanation, the example and the practice, as it would be anywhere else.

Support, and how it is taken away

- The worked example is on the early activities of a station and not on the late ones. That fade is deliberate.
- The hint is a ladder asked for one rung at a time, and the top of it is still the technique on different data, never the answer.
- Read aloud is on every task.

- Prescribed values are boxed and in a monospace face as well as coloured, so they survive a black-and-white printout and a high-contrast mode.
- **Lower literacy and EAL:** pre-teach the vocabulary (Variable, Constant, Casting, Concatenation, Integer division, Modulus, Exponent, Pseudo-random, Selection, Condition, elif, Nested selection). Accept the key fact spoken before it is written, and in two or three words.
- **SEND:** pace is not fixed. Progress saves after every answer and reopens at the earliest unfinished question, so a pupil may finish a station next lesson without losing anything.
- A headset is never required.

Checking, and what to do with what you find

Three checks, in this order: the starter (did last lesson stick), the hinge question (is the class ready for independent work), the exit question (did this lesson stick). The first two change what you do in the lesson; the third changes what you start the next one with.

When a hinge question splits the class, go back to the station it sits on and work its example again, aloud, before anyone goes on. When a test comes back, the reflection sheet names the topics to revisit and review mode on those lessons reopens exactly the questions a pupil got wrong.

The Python sequence, and why the order is compulsory

Every station releases the same way:

Learn → See → Predict → Change → Complete → Build → Apply.

- **Learn and See:** a sentence and a worked example, then a program to run. Nothing is marked.
- **Predict:** say what it will display before running it. This is where a misreading surfaces cheaply.
- **Change:** one thing to alter in a working program.
- **Complete:** a program with a gap in it.
- **Fix:** a program with a real error and a real traceback.
- **Build:** from nothing, with the brief and the hint ladder.
- **Apply:** the Challenge, the hardest task on the station.

A pupil works through them in order and completes every one, because each teaches what the next needs: a pupil who skips Predict arrives at Build without having found out that they read the program wrongly. When they are stuck they get the hint ladder, and then they ask you. They do not skip the question, and there is no teacher bypass.

What this costs is time, and it is the right thing to spend it on. What it buys is that a pupil who reaches lesson 13 has written every program between here and there.

Where this sits against the specification

The Python course is not an OCR topic and carries no specification reference. It teaches programming in a real language, which Component 02 assesses the thinking behind: a pupil who has written, run and fixed their own programs reads an exam question about iteration differently from one who has only read about it. Where it supports a topic, that topic is named. Where it goes beyond the specification - the editor, the test-based marking, SQL through sqlite3 - it is named as enrichment, not as coverage.

- **2.1 Algorithms** — the searching and sorting algorithms of lesson 11, written and run rather than traced on paper
- **2.2 Programming fundamentals** — every programming technique the topic names: variables and constants, the three programming constructs, data types and casting, string handling, arrays, file handling, records, SQL and subprograms
- **2.3 Producing robust programs** — validation and authentication in lesson 12, and the habit of reading a failing test rather than guessing

Beyond the specification, and named as such:

- The in-browser editor and the Python runtime behind it.
- Marking by running a pupil's program against test cases.

- sqlite3 as the way SQL is run, which is an implementation choice: the specification asks for SQL, not for a particular database library.
- The compulsory question order and the save-and-resume progression.

Where the approach comes from

The shape of a lesson here — retrieve, explain, model, practise with support that fades, check, apply, review — is ordinary evidence-informed practice rather than anything this product invented. Useful starting points, none of them about Revise 360:

- Rosenshine, *Principles of Instruction* (American Educator, 2012) — the daily review, small steps, worked examples and checking for understanding that this structure follows.
- Sweller, van Merriënboer and Paas on cognitive load and the worked-example effect — why the example comes before the independent task and why the support is withdrawn gradually.
- Dunlosky et al., *Improving Students' Learning With Effective Learning Techniques* (2013) — the evidence for retrieval practice and spacing, which the starters and the review mode use.
- Wiliam, *Embedded Formative Assessment* — hinge questions and what to do with the answers.

What is not claimed. Nothing above is evidence that Revise 360 improves outcomes. No trial of this product has been run, and immersion is not claimed to improve learning by itself. The citations are for the teaching structure; whether these resources work in your classroom is a question your own assessment answers.

Revise 360 — a product of Revise 360 Ltd, created by Olly Pettitt. An independent resource: not written, endorsed or approved by OCR or any exam board. Classroom use and personal study only.